

AI Assisted Coding

Assignment – 7.1

Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

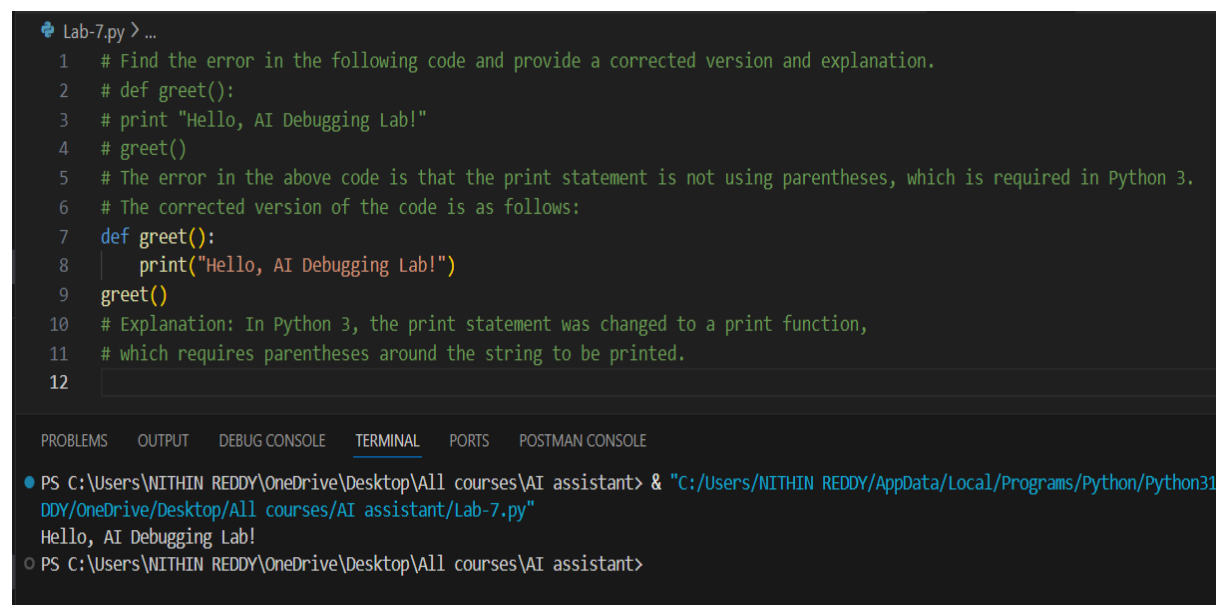
```
def greet():
```

```
print "Hello, AI Debugging Lab!"
```

```
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.



```
Lab-7.py > ...
1 # Find the error in the following code and provide a corrected version and explanation.
2 # def greet():
3 # print "Hello, AI Debugging Lab!"
4 # greet()
5 # The error in the above code is that the print statement is not using parentheses, which is required in Python 3.
6 # The corrected version of the code is as follows:
7 def greet():
8     print("Hello, AI Debugging Lab!")
9     greet()
10 # Explanation: In Python 3, the print statement was changed to a print function,
11 # which requires parentheses around the string to be printed.
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
● PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python311/Python311.exe" C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant\Lab-7.py
Hello, AI Debugging Lab!
○ PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):
```

```
    if n = 10:
```

```
        return "Ten"
```

```
    else:
```

```
        return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

```
12
13 #fix the error in the if condition for the below provided code and explain the error and the fix.
14 # def check_number(n):
15 #     if n = 10:
16 #         return "Ten"
17 #     else:
18 #         return "Not Ten"
19 # The error in the above code is that the assignment operator (=) is used instead of the equality operator (==) in the if condition.
20 # The corrected version of the code is as follows:
21 ✓ def check_number(n):
22 ✓     if n == 10:
23         return "Ten"
24 ✓     else:
25         return "Not Ten"
26 # Explanation: In the original code, the if condition is trying to assign the value 10 to n instead of checking if n is equal to 10.
27 # This will result in a syntax error. In the corrected version, we use the equality operator (==) to check if n is equal to 10,
28 # which is the intended behavior.
29
```

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

```
print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

```
30
31 # Handle the error in the below code by try and except block and explain the error and add the user friendly error message.
32 # def read_file(filename):
33 #     with open(filename, 'r') as f:
34 #         return f.read()
35 # print(read_file("nonexistent.txt"))
36 # The error in the above code is that it tries to read a file that does not exist, which will raise a FileNotFoundError.
37 # The corrected version of the code with error handling is as follows:
38 def read_file(filename):
39     try:
40         with open(filename, 'r') as f:
41             return f.read()
42     except FileNotFoundError:
43         return f"Error: The file '{filename}' was not found. Please check the filename and try again."
44 print(read_file("nonexistent.txt"))
45 # Explanation: In the original code, if the file "nonexistent.txt" does not exist,
46 # it will raise a FileNotFoundError and the program will crash.
47 # In the corrected version, we wrap the file reading operation in a try-except block to catch the FileNotFoundError.
48 # If the file is not found, we return a user-friendly error message that informs the user about the issue and suggests checking the filename.
49 # This way, the program does not crash and provides helpful feedback to the user.
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE Python + ▢

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/NITHIN REDDY/Desktop/All courses/AI assistant/Lab-7.py"
Error: The file 'nonexistent.txt' was not found. Please check the filename and try again.
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

```
class Car:
```

```
    def start(self):
```

```
        return "Car started"
```

```
my_car = Car()
```

```
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method or correct the method call.

- Use 3 assert tests to confirm the corrected class works.

```

51 #In the below code fix the bug related to the Calling an undefined method and explain the error and the fix.
52 # class Car:
53 # def start(self):
54 # return "Car started"
55 # my_car = Car()
56 # print(my_car.drive()) # drive() is not defined
57 # The error in the above code is that the method `drive()` is called on the `my_car` object, but it is not defined in the `Car` class.
58 # The corrected version of the code is as follows:
59 class Car:
60     def start(self):
61         return "Car started"
62
63     def drive(self):
64         return "Car is driving"
65 my_car = Car()
66 print(my_car.drive()) # Now drive() is defined
67 # Explanation: In the original code, the `drive()` method is called on the `my_car` object,
68 # but since it is not defined in the `Car` class, it will raise an AttributeError.
69 # In the corrected version, we define the `drive()` method within the `Car` class,
70 # which allows us to call it on the `my_car` object without any errors.

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python +

```

PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/python.exe" "c:/Users/NITHIN REDDY/Desktop/All courses/AI assistant/Lab-7.py"
Car is driving
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>

```

Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```

def add_five(value):
    return value + 5

print(add_five("10"))

```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

```
71
72 #In the below code there is a TypeError due to mixing string and integer. You fix the error and add that numbers.
73 # def add_five(value):
74 # return value + 5
75 # print(add_five("10"))
76 # The error in the above code is that it tries to add an integer (5) to a string ("10"), which will raise a TypeError.
77 # The corrected version of the code is as follows:
78 def add_five(value):
79     try:
80         return int(value) + 5
81     except ValueError:
82         return f"Error: The value '{value}' is not a valid number. Please provide a numeric value."
83 print(add_five("10")) # This will now return 15
84 # Explanation: In the original code, the function `add_five` attempts to add an integer (5) to a string ("10"),
85 # which is not allowed in Python and results in a TypeError.
86 # In the corrected version, we convert the input `value` to an integer using `int(value)`.
87
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant> & "C:/Users/NITHIN REDDY/AppData/Local/Programs/Python/Python313/py
ive/Desktop/All courses/AI assistant/Lab-7.py"
```

```
15
```

```
PS C:\Users\NITHIN REDDY\OneDrive\Desktop\All courses\AI assistant>
```